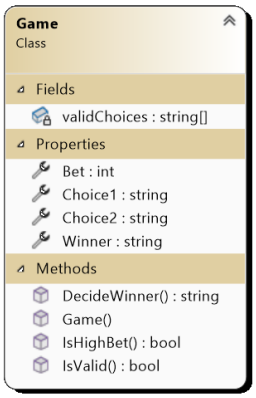
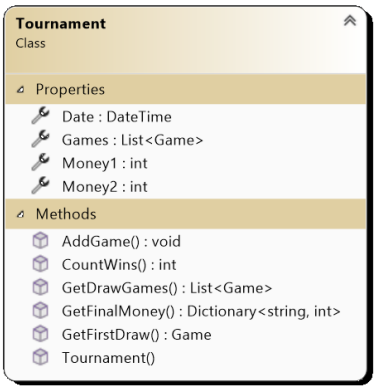
Kő, papír, olló – 22 pont

A kő, papír, olló népszerű kétszemélyes játék. Egy mérkőzés során mindkét játékos választ egyet a kő, a papír és az olló tárgyak közül. A győztes az, akinek a tárgya le tudja győzni a másik által választott tárgyat. A szabályok szerint a `kő > olló`, a `papír > kő`, és az `olló > papír`. Ha a két játékos ugyanazt a tárgyat választja, akkor döntetlen az eredmény.



Anna és Béla egy kő, papír, olló bajnokságon vesznek részt, amelyhez kapcsolódóan egy **`Game`** és egy **`Tournament`** osztály található a kezdő projektben. (A két név szándékosan beégetett a projektbe, csak ketten vesznek részt a bajnokságon.)





1. Készíts egy **NUnit** teszt projektet **`RockPaperScissorsTests`** néven, azon belül pedig egy **`GameTests`** (2-4. feladat) és egy **`TournamentTests`** (5-11. feladat) teszt osztályt! **1 pont**

A további feladatok során egy-egy függvényhez kell tesztet írni. **Minden tesztelésre szolgáló függvény neve a tesztelendő függvény nevével kezdődjön!**

Például, ha tesztelni kell az `IsEven()` függvényt, akkor a tesztelő függvény lehet `IsEven_Test()` vagy `IsEven_True()` stb…

A teljes pontszámhoz minden feladatban **100%**-os **branch coverage** az elvárás!

2. Az `IsValid()` **statikus** függvény egy szöveget pár paraméterként, és megadja, hogy létezik-e ilyen választási lehetőség a kő, papír, olló játékban!

   Teszteld le mindhárom helyes szövegre, és legalább két nem választható lehetőségre is! **2 pont**

3. A szintén **statikus** `DecideWinner()` függvény két szöveget vár paraméterként: az első Anna, a második Béla választása. Visszatérési értéke a győztes neve, illetve egyforma választás esetén a „Döntetlen” szöveg. **3 pont**

4. Teszteld a **`Game`** osztály konstruktorát! Helyes működés esetén ellenőrizd mindegyik property értékét, helytelen meghíváskor pedig azt, hogy megfelelő kivételt dob-e a konstruktor! **3 pont**

5. Teszteld a **`Tournament`** osztály konstruktorát, hogy helyesen állítja-e be a kezdő pénzeket és a játékokat tároló listát! **1 pont**

6. Oldd meg, hogy a további tesztek mindegyike előtt létrejöjjön egy **`Tournament`** objektum, amely a következő játékokat tartalmazza: **2 pont**

```
Game g1 = new("kő", "papír", 100); // Béla nyert
Game g2 = new("olló", "olló", 200); // Döntetlen
Game g3 = new("kő", "olló", 500); // Anna nyert
Game g4 = new("papír", "kő", 200); // Anna nyert
Game g5 = new("kő", "kő", 300); // Döntetlen
```

   Ügyelj rá, hogy minden objektum (a bajnokság és a játékok is) használható legyen a tesztek során, de a teszt osztályon kívül ne legyenek láthatók!

7. Teszteld a `CountWins()` metódust **mindkét játékossal**! **2 pont**

8. Teszteld a `GetFirstDraw()` metódust olyan bajnoksággal, amikor **volt** döntetlen és olyannal is, amikor **nem volt**! **2 pont**

9. Írj legalább egy tesztet a `GetDrawGames()` metódusra! **1 pont**

10. A `GetFinalMoney()` feladata, hogy a `Games` lista alapján meghatározza, mennyi pénze marad a bajnokság végére Annának és Bélának. Visszatér egy szótárral, amelynek kulcsai a két játékos neve, értékei pedig a pénzük. Pótold a függvény hiányzó részeit a kommenteknek megfelelően! **3 pont**

11. Teszteld a `GetFinalMoney()` metódust! **2 pont**